

Artificial Intelligence Lab Assignment - 2

Roll- 20262603016

Name - Sagar Dhankar

Pseudo Code

START

FUNCTION tokenize_sentences(text):

 SPLIT text using ".", "!", "?" as separators

 FOR each sentence:

 CONVERT sentence to lowercase

 REMOVE characters other than letters, numbers, and spaces

 REPLACE multiple spaces with one space

 REMOVE leading/trailing spaces

 IF sentence is not empty:

 ADD sentence to result

 RETURN result

FUNCTION levenshtein_distance(sentence1, sentence2):

 SPLIT both sentences into words

 CREATE DP table

 INITIALIZE first column with deletion costs

 INITIALIZE first row with insertion costs

 FOR each word in sentence1:

 FOR each word in sentence2:

 IF words are equal:

 change_cost = 0

```
ELSE:  
    change_cost = 1
```

```
dp[i][j] = MIN(  
    deletion,  
    insertion,  
    replacement  
)
```

```
RETURN final DP value
```

```
FUNCTION astar_alignment(doc1, doc2):
```

```
    SET initial position = (0, 0)  
    CREATE priority queue  
    CREATE distance cache
```

```
    heuristic(i, j):  
        RETURN absolute difference between  
        remaining sentences in both documents
```

```
    ADD initial state to priority queue
```

```
    WHILE priority queue is not empty:
```

```
        REMOVE state with lowest f-cost
```

```
        IF this state has already been reached  
        with a lower cost:  
            CONTINUE
```

```
        IF both documents are completely processed:  
            RETURN alignment path and total cost
```

```
        IF sentences remain in both documents:  
            CALCULATE Levenshtein distance  
            CREATE MATCH operation  
            ADD new state to priority queue
```

```
        IF sentence remains in document 1:  
            CALCULATE number of words  
            CREATE SKIP_DOC1 operation  
            ADD new state to priority queue
```

```
IF sentence remains in document 2:  
    CALCULATE number of words  
    CREATE SKIP_DOC2 operation  
    ADD new state to priority queue
```

```
RETURN empty path and infinity
```

```
FUNCTION detect_plagiarism(text1, text2, threshold):
```

```
    TOKENIZE both documents into sentences
```

```
    ALIGN sentences using A*
```

```
    FOR every alignment operation:
```

```
        IF operation is not MATCH:  
            CONTINUE
```

```
        GET matched sentences
```

```
        CALCULATE maximum sentence length
```

```
        similarity = 1 - (edit_distance / length)
```

```
        STORE:  
            sentence 1  
            sentence 2  
            edit distance  
            similarity  
            plagiarism status
```

```
    RETURN results and total alignment cost
```

```
FUNCTION display_results(results, total_cost):
```

```
    PRINT total alignment cost
```

```
    FOR each result:  
        PRINT both sentences  
        PRINT edit distance  
        PRINT similarity
```

```
IF similarity >= threshold:
    PRINT "POTENTIAL PLAGIARISM"
ELSE:
    PRINT "No significant similarity"
```

```
FUNCTION run_tests():
```

```
    RUN four plagiarism test cases:
        1. Identical documents
        2. Slightly modified documents
        3. Completely different documents
        4. Partially overlapping documents
```

```
    DISPLAY results for each test
```

```
MAIN:
    CALL run_tests()
```

```
END
```

Python Code

```
import re
import heapq
```

```
# -----
# 1. Sentence tokenization
# -----
def tokenize_sentences(text):
    """
    Split text into cleaned sentences.
    - Split on '.', '!', '?'
    - Lowercase
    - Remove non letter/number/space characters
```

- Collapse multiple spaces
- Strip leading/trailing spaces
- Drop empty sentences

```
"""
```

```
raw_sentences = re.split(r'[!?!]', text)
```

```
result = []
```

```
for sentence in raw_sentences:
```

```
    sentence = sentence.lower()
```

```
    sentence = re.sub(r'^[a-z0-9\s]', '', sentence)
```

```
    sentence = re.sub(r'\s+', ' ', sentence)
```

```
    sentence = sentence.strip()
```

```
    if sentence:
```

```
        result.append(sentence)
```

```
return result
```

```
# -----
```

```
# 2. Levenshtein distance (word-level)
```

```
# -----
```

```
def levenshtein_distance(sentence1, sentence2):
```

```
    """
```

```
    Compute word-level edit distance between two sentences.
```

```
    """
```

```
    words1 = sentence1.split()
```

```
    words2 = sentence2.split()
```

```
    n, m = len(words1), len(words2)
```

```
    dp = [[0] * (m + 1) for _ in range(n + 1)]
```

```
    # First column: deletion costs
```

```
    for i in range(n + 1):
```

```
        dp[i][0] = i
```

```
    # First row: insertion costs
```

```
    for j in range(m + 1):
```

```
        dp[0][j] = j
```

```
    for i in range(1, n + 1):
```

```
        for j in range(1, m + 1):
```

```
            if words1[i - 1] == words2[j - 1]:
```

```
                change_cost = 0
```

```

    else:
        change_cost = 1

    deletion = dp[i - 1][j] + 1
    insertion = dp[i][j - 1] + 1
    replacement = dp[i - 1][j - 1] + change_cost

    dp[i][j] = min(deletion, insertion, replacement)

return dp[n][m]

# -----
# 3. A* based sentence alignment
# -----
def astar_alignment(doc1, doc2):
    """
    Align sentences of doc1 and doc2 using A* search.

    Operations:
    MATCH    - align doc1[i] with doc2[j]
    SKIP_DOC1 - skip a sentence in doc1 (treated as unmatched/deleted)
    SKIP_DOC2 - skip a sentence in doc2 (treated as unmatched/inserted)

    Returns:
    (path, total_cost)
    path is a list of operation dicts in order.
    """
    len1, len2 = len(doc1), len(doc2)

    def heuristic(i, j):
        remaining1 = len1 - i
        remaining2 = len2 - j
        return abs(remaining1 - remaining2)

    # Priority queue entries: (f_cost, g_cost, i, j, path)
    start = (heuristic(0, 0), 0, 0, 0, [])
    open_heap = [start]

    # best known cost to reach a given (i, j) state
    best_cost = {(0, 0): 0}

    counter = 0 # tie-breaker for heap comparisons

```

```

while open_heap:
    f_cost, g_cost, i, j, path = heapq.heappop(open_heap)

    # Skip stale entries
    if g_cost > best_cost.get((i, j), float('inf')):
        continue

    # Goal check
    if i == len1 and j == len2:
        return path, g_cost

    # MATCH operation
    if i < len1 and j < len2:
        edit_dist = levenshtein_distance(doc1[i], doc2[j])
        new_g = g_cost + edit_dist
        new_state = (i + 1, j + 1)

        if new_g < best_cost.get(new_state, float('inf')):
            best_cost[new_state] = new_g
            new_path = path + [{
                'operation': 'MATCH',
                'doc1_index': i,
                'doc2_index': j,
                'sentence1': doc1[i],
                'sentence2': doc2[j],
                'cost': edit_dist
            }]
            new_f = new_g + heuristic(i + 1, j + 1)
            counter += 1
            heapq.heappush(open_heap, (new_f, new_g, i + 1, j + 1, new_path))

    # SKIP_DOC1 operation
    if i < len1:
        skip_cost = len(doc1[i].split())
        new_g = g_cost + skip_cost
        new_state = (i + 1, j)

        if new_g < best_cost.get(new_state, float('inf')):
            best_cost[new_state] = new_g
            new_path = path + [{
                'operation': 'SKIP_DOC1',
                'doc1_index': i,
                'doc2_index': None,
                'sentence1': doc1[i],

```

```

        'sentence2': None,
        'cost': skip_cost
    }}
    new_f = new_g + heuristic(i + 1, j)
    counter += 1
    heapq.heappush(open_heap, (new_f, new_g, i + 1, j, new_path))

# SKIP_DOC2 operation
if j < len2:
    skip_cost = len(doc2[j].split())
    new_g = g_cost + skip_cost
    new_state = (i, j + 1)

    if new_g < best_cost.get(new_state, float('inf')):
        best_cost[new_state] = new_g
        new_path = path + [{
            'operation': 'SKIP_DOC2',
            'doc1_index': None,
            'doc2_index': j,
            'sentence1': None,
            'sentence2': doc2[j],
            'cost': skip_cost
        }]
        new_f = new_g + heuristic(i, j + 1)
        counter += 1
        heapq.heappush(open_heap, (new_f, new_g, i, j + 1, new_path))

return [], float('inf')

```

```

# -----
# 4. Plagiarism detection
# -----
def detect_plagiarism(text1, text2, threshold=0.7):
    """
    Detect plagiarism between two texts by tokenizing into sentences,
    aligning them with A*, and computing similarity per matched pair.
    """
    doc1 = tokenize_sentences(text1)
    doc2 = tokenize_sentences(text2)

    path, total_cost = astar_alignment(doc1, doc2)

    results = []

```

```

for op in path:
    if op['operation'] != 'MATCH':
        continue

    sentence1 = op['sentence1']
    sentence2 = op['sentence2']
    edit_distance = op['cost']

    words1 = sentence1.split()
    words2 = sentence2.split()
    max_len = max(len(words1), len(words2))

    if max_len == 0:
        similarity = 1.0
    else:
        similarity = 1 - (edit_distance / max_len)

    is_plagiarism = similarity >= threshold

    results.append({
        'sentence1': sentence1,
        'sentence2': sentence2,
        'edit_distance': edit_distance,
        'similarity': similarity,
        'is_plagiarism': is_plagiarism
    })

return results, total_cost

```

```

# -----
# 5. Display results
# -----
def display_results(results, total_cost, threshold=0.7):
    print(f"Total alignment cost: {total_cost}")
    print("-" * 60)

    for result in results:
        print(f"Sentence 1: {result['sentence1']}")
        print(f"Sentence 2: {result['sentence2']}")
        print(f>Edit distance: {result['edit_distance']}")
        print(f>Similarity: {result['similarity']:.2f}")

```

```

    if result['similarity'] >= threshold:
        print("Status: POTENTIAL PLAGIARISM")
    else:
        print("Status: No significant similarity")

print("-" * 60)

# -----
# 6. Test runner
# -----
def run_tests():
    threshold = 0.7

    test_cases = [
        {
            'name': 'Identical documents',
            'text1': "The quick brown fox jumps over the lazy dog. Python is a great language.",
            'text2': "The quick brown fox jumps over the lazy dog. Python is a great language."
        },
        {
            'name': 'Slightly modified documents',
            'text1': "The quick brown fox jumps over the lazy dog. Python is a great language.",
            'text2': "The quick brown fox leaps over the lazy dog. Python is a wonderful language."
        },
        {
            'name': 'Completely different documents',
            'text1': "The stock market crashed yesterday due to inflation fears.",
            'text2': "My cat enjoys sleeping on the warm windowsill all day."
        },
        {
            'name': 'Partially overlapping documents',
            'text1': "Machine learning is a subset of artificial intelligence. It uses data to learn
patterns. Many industries use it today.",
            'text2': "Deep learning is a subset of machine learning. Many industries use it today. It
requires large amounts of data."
        }
    ]

    for idx, case in enumerate(test_cases, start=1):
        print("=" * 60)
        print(f"TEST CASE {idx}: {case['name']}")
        print("=" * 60)

```

```
results, total_cost = detect_plagiarism(case['text1'], case['text2'], threshold)
display_results(results, total_cost, threshold)
print()
```

```
# -----
# MAIN
# -----
if __name__ == "__main__":
    run_tests()
```